

Basic:

1. What is Apache Airflow, and why is it used?

Answer: Apache Airflow is an open-source workflow automation and orchestration tool that allows users to programmatically author, schedule, and monitor workflows as Directed Acyclic Graphs (DAGs). It is used to manage data pipelines, automate ETL processes, and ensure reliable task execution with built-in retry mechanisms, logging, and monitoring.

2. What are DAGs in Airflow?

Answer: A DAG (Directed Acyclic Graph) in Airflow is a collection of tasks that are organized and executed with dependencies. DAGs define the structure of a workflow, ensuring tasks execute in the correct order without cycles.

3. How do you define a DAG in Apache Airflow?

Answer: DAGs are defined using Python scripts. Here's a basic example:

```
from airflow import DAG

from airflow.operators.dummy import DummyOperator

from datetime import datetime
```

```
default_args = {
    'owner': 'airflow',
    'start_date': datetime(2024, 2, 1),
    'retries': 1
}
```

```
dag = DAG(
    'example_dag',
    default_args=default_args,
    schedule_interval='@daily'
)
```

```
task1 = DummyOperator(task_id='start', dag=dag)
task2 = DummyOperator(task_id='end', dag=dag)
task1 >> task2 # Setting dependencies
```

4. What is an Operator in Airflow? Name some commonly used Operators.

Answer: Operators in Airflow define tasks and determine what operations are executed. Commonly used operators include:

- **BashOperator:** Executes a Bash command.
- **PythonOperator:** Runs a Python function.
- **PostgresOperator/MySqlOperator:** Executes SQL queries.
- **DummyOperator:** Used as a placeholder or for structuring DAGs.
- **Sensor Operators:** Wait for external conditions like file arrival.

5. What is the difference between an Operator and a Task?

Answer: An **Operator** is a template that defines a specific type of task (e.g., BashOperator, PythonOperator). A **Task** is an instantiation of an operator within a DAG.

6. How do you schedule a DAG in Apache Airflow?

Answer: DAGs are scheduled using the `schedule_interval` parameter. Example intervals:

- `@daily` – Runs once per day.
- `@hourly` – Runs every hour.
- `'0 12 * * *'` – Runs daily at noon (CRON syntax).

7. What are Task Dependencies in Airflow, and how do you define them?

Answer: Task dependencies define the order of execution. They can be set using `>>` (downstream) or `<<` (upstream):

`task1 >> task2` # task2 runs after task1

`task3 << task2` # task3 runs after task2

8. What is the difference between sequential and parallel execution of tasks?

Answer:

- **Sequential Execution:** Tasks execute one after another.
- **Parallel Execution:** Tasks execute simultaneously if no dependencies prevent them.

9. What is the Airflow Web UI, and what can you do with it?

Answer: The Airflow Web UI provides a graphical interface to:

- View DAG runs
- Trigger DAGs manually
- Monitor task logs
- Pause/unpause DAGs
- Check execution history

10. How do you manually trigger a DAG in Apache Airflow?

Answer: You can trigger a DAG using the Web UI or CLI:

```
airflow dags trigger example_dag
```

11. What are XComs in Airflow, and how do they work?

Answer: XComs (Cross-Communication) allow tasks to exchange data. Example usage:

```
task_instance.xcom_push(key='data_key', value='some_value')  
value = task_instance.xcom_pull(task_ids='previous_task', key='data_key')
```

12. How do you pass data between tasks in a DAG?

Answer: Using **XComs** or **return values** in the PythonOperator.

13. How does Airflow handle retries for failed tasks?

Answer: Airflow allows retry configuration using `retries` and `retry_delay` in `default_args`.

```
default_args = {'retries': 3, 'retry_delay': timedelta(minutes=5)}
```

14. What are Task Instances in Airflow?

Answer: A Task Instance is a specific execution of a task within a DAG run.

15. How do you pause and unpause DAGs in Airflow?

Answer: You can do it via the Web UI or CLI:

```
airflow dags pause example_dag
```

```
airflow dags unpause example_dag
```

16. What are the default Airflow Executors, and how do they differ?

Answer:

- **SequentialExecutor:** Runs tasks one at a time.
- **LocalExecutor:** Runs multiple tasks in parallel on the same machine.
- **CeleryExecutor:** Uses distributed workers for parallel execution.
- **KubernetesExecutor:** Runs each task in a separate Kubernetes pod.

17. How do you store logs in Airflow?

Answer: Logs are stored in the `logs/` directory by default but can be configured to be stored in **Amazon S3, Google Cloud Storage, or a database**.

18. How do you set environment variables in Airflow?

Answer: You can set environment variables in the Airflow configuration file (`airflow.cfg`) or by exporting them:

```
export AIRFLOW_HOME=/path/to/airflow
```

```
export AIRFLOW__CORE__DAGS_FOLDER=/path/to/dags
```

Intermediate:

1. What is the difference between Airflow's LocalExecutor, CeleryExecutor, and KubernetesExecutor?

Answer:

- **LocalExecutor:** Runs tasks in parallel on the same machine, ideal for small-scale deployments.
- **CeleryExecutor:** Distributes tasks across multiple worker nodes using Celery, suited for production.
- **KubernetesExecutor:** Runs each task in a separate Kubernetes pod, ensuring high scalability and isolation.

2. How does Airflow's backfill mechanism work?

Answer: Backfilling allows running historical DAG runs for a specified date range. Use the CLI:

```
airflow dags backfill -s 2024-01-01 -e 2024-01-10 my_dag
```

This helps rerun past executions for consistency.

3. What is the role of the Airflow Scheduler?

Answer: The Airflow Scheduler monitors DAGs, determines task execution order, and triggers tasks based on their dependencies and schedules.

4. How do you create a custom Operator in Airflow?

Answer: Extend the BaseOperator class and implement execute():

```
from airflow.models import BaseOperator
```

```
class MyCustomOperator(BaseOperator):
```

```
    def execute(self, context):
```

```
        print("Executing my custom operator!")
```

5. What are Airflow Hooks, and how do they work?

Answer: Hooks provide interfaces for external systems (e.g., databases, APIs). Example:

```
from airflow.hooks.postgres_hook import PostgresHook
```

```
hook = PostgresHook(postgres_conn_id='my_conn_id')
```

```
records = hook.get_records("SELECT * FROM my_table;")
```

6. How do you use Sensors in Apache Airflow?

Answer: Sensors wait for conditions before proceeding. Example FileSensor:

```
from airflow.sensors.filesystem import FileSensor
```

```
sensor = FileSensor(task_id='check_file', filepath='/path/to/file', poke_interval=30, timeout=600, dag=dag)
```

7. How does Airflow manage dependencies between tasks?

Answer: By using >> and << operators or set_upstream() and set_downstream() methods.

8. What is the role of the Metadata Database in Airflow?

Answer: Stores DAG runs, task statuses, and other metadata. Uses relational databases like PostgreSQL or MySQL.

9. How do you use Branching in Apache Airflow?

Answer: Use BranchPythonOperator to dynamically choose the execution path:

```
from airflow.operators.python import BranchPythonOperator

def choose_branch():
    return 'task_a' if condition else 'task_b'

branch = BranchPythonOperator(task_id='branching', python_callable=choose_branch,
dag=dag)
```

Advanced:

1. How do you deploy Apache Airflow in a production environment?

Answer: Best practices include:

- Using **CeleryExecutor** or **KubernetesExecutor** for scalability.
- Storing metadata in PostgreSQL.
- Configuring persistent log storage (e.g., S3, GCS).
- Setting up monitoring with **Prometheus & Grafana**.

2. What are the best practices for scaling Apache Airflow?

Answer:

- Use **Celery/Kubernetes Executor** for distributed execution.
- Optimize DAG performance by reducing task execution time.
- Increase worker node capacity as required.

3. How do you secure an Airflow deployment?

Answer:

- Use **RBAC (Role-Based Access Control)** for permissions.
- Enable **TLS encryption** for Web UI & APIs.
- Store sensitive configurations in environment variables or **Secret Managers**.

4. How do you monitor DAG performance in Apache Airflow?

Answer:

- Use **Airflow's built-in logs & UI**.
- Configure **Prometheus + Grafana** for real-time monitoring.
- Enable **task duration metrics**.

5. How does Airflow handle concurrency and parallelism?

Answer:

- **dag_concurrency**: Number of parallel tasks per DAG.
- **parallelism**: Total tasks across all DAGs.
- **max_active_runs**: Limits DAG instances running concurrently.

6. How do you use Airflow with Apache Kafka for real-time workflows?

Answer:

- Use a **Sensor Operator** to monitor Kafka topics.
- Integrate **Apache Spark** for real-time ETL processing.

7. How do you implement event-driven workflows in Apache Airflow?

Answer:

- Use **Trigger DAG Runs** based on external events.
- Leverage **Webhook APIs** or **Message Queues (Kafka, RabbitMQ)**.

8. How do you use Airflow to automate machine learning workflows?

Answer:

- Define ML pipelines as DAGs.
- Use **PythonOperator** to trigger model training & evaluation.
- Store models in **MLflow** or **S3**.

9. How do you troubleshoot slow DAG execution times?

Answer:

- Optimize task execution time.
- Increase **worker node capacity**.
- Reduce **XCom usage**, as it impacts performance.

10. What is the difference between Astronomer and Open-Source Airflow?

Answer:

- **Open-Source Airflow**: Community-supported, self-managed.
- **Astronomer**: Enterprise solution with **managed Airflow, monitoring, security, and scalability**.